

Módulo 1 - Teoría

Curso Spring Boot 2026

Índice

Módulo 1 - Introducción a Spring Boot y arquitectura back-end	4
Propósito del módulo	4
Punto 1.1 - Qué es Spring Boot y su papel en aplicaciones modernas	5
Objetivos de aprendizaje	5
Bloque 1 - Qué es Spring Boot y por qué existe	5
1.1 El problema no empieza en la lógica de negocio	5
1.2 Convención frente a configuración manual	6
1.3 Lo que Spring Boot no hace por ti	6
Pregunta	6
Respuesta razonada	6
1.4 Las tres ideas clave para empezar	7
Pregunta	7
Respuesta razonada	7
Bloque 2 - Spring Framework y Spring Boot	7
2.1 Spring Framework es la base	7
2.2 Del Spring altamente configurado al arranque opinado	7
2.3 Separar capacidad de política de arranque	8
Pregunta	8
Respuesta razonada	8
Pregunta	8
Respuesta razonada	8
Bloque 3 - Auto-configuración	8
3.1 Qué significa realmente “automática”	8
3.2 @SpringBootApplication y la auto-configuración	9
3.3 Condiciones positivas y negativas	9
3.4 Ejemplo conceptual: aplicación web	9
3.5 Back-off: la configuración automática debe ceder	10
Pregunta	10
Respuesta razonada	10
3.6 Diagnóstico: no memorizar cientos de auto-configuraciones	10
Bloque 4 - Starters y gestión coherente de dependencias	10
4.1 Qué es un starter	10
4.2 Dependencias directas y transitivas	11
4.3 Starter no significa “usar todo en cada petición”	11
4.4 Versiones coordinadas	11
Pregunta	11
Respuesta razonada	11
4.5 Starter frente a dependencia específica	11
Bloque 5 - Servidor embebido y aplicaciones modernas	12
5.1 Dos modelos de despliegue	12
5.2 “Embebido” no significa que Tomcat deje de existir	12
5.3 Ventajas del modelo embebido	12
5.4 Costes y límites	12
5.5 APIs, servicios y contenedores	13

5.6 Monolito y microservicio no son sinónimos de viejo y moderno	13
Pregunta	13
Respuesta razonada	13
Cómo se combinan las tres ideas en nuestro proyecto	13
Resumen del Punto 1.1	13
Punto 1.2 - Arquitectura cliente-servidor y HTTP	14
Objetivos de aprendizaje	14
Bloque 1 - Cliente y servidor	14
1.1 Dos papeles, no necesariamente dos tipos de máquina	14
1.2 localhost, dirección y puerto	15
1.3 Una URL no identifica un método Java	15
Pregunta	15
Respuesta razonada	15
Bloque 2 - El ciclo petición-respuesta	15
2.1 HTTP organiza una conversación	15
2.2 Qué contiene una petición	16
2.3 Qué contiene una respuesta	16
2.4 Del socket al controlador	16
Pregunta	17
Respuesta razonada	17
Bloque 3 - Front-end y back-end	17
3.1 Responsabilidades diferentes	17
3.2 Por qué separar	17
3.3 API como frontera	18
Pregunta	18
Respuesta razonada	18
Bloque 4 - Métodos HTTP y códigos de estado	18
4.1 El método forma parte del endpoint	18
4.2 Métodos principales	18
4.3 Familias de códigos de estado	18
4.4 Códigos que usaremos pronto	19
4.5 404 no es 405	19
Pregunta	19
Respuesta razonada	19
Bloque 5 - Herramientas para observar HTTP	19
5.1 El navegador: útil, pero interpreta por ti	19
5.2 DevTools: el navegador se vuelve observable	20
5.3 curl: cliente mínimo y reproducible	20
5.4 La salida exacta puede cambiar sin cambiar el concepto	20
5.5 Diagnóstico por capas	21
Pregunta	21
Respuesta razonada	21
Resumen del Punto 1.2	21
Punto 1.3 - JSON y Jackson	21
Objetivos de aprendizaje	21
Bloque 1 - Qué es JSON	22
1.1 Un formato de intercambio de datos	22
1.2 Por qué encaja bien en APIs	22
1.3 JSON frente a un objeto Java	22
1.4 JSON no transporta comportamiento	23
Pregunta	23
Respuesta razonada	23
Bloque 2 - Sintaxis de JSON	23
2.1 Objetos y pares clave-valor	23
2.2 Arrays	23
2.3 Tipos de valor	24

2.4 Errores frecuentes	24
2.5 JSON no admite comentarios estándar	24
Pregunta	24
Respuesta razonada	24
Bloque 3 - Convenciones de JSON en APIs REST	25
3.1 Nombres predecibles	25
3.2 Fechas: contrato, no tipo nativo	25
3.3 null, ausencia y listas vacías no significan lo mismo	25
3.4 DTOs anidados	25
3.5 Compatibilidad y propiedades desconocidas	26
Pregunta	26
Respuesta razonada	26
Bloque 4 - Jackson	26
4.1 Serialización y deserialización	26
4.2 Cuando un controlador devuelve un DTO	26
4.3 Cuando usamos @RequestBody	27
4.4 Constructor y propiedades	27
4.5 Jackson no valida reglas de negocio	27
Pregunta	27
Respuesta razonada	27
Bloque 5 - Anotaciones de Jackson	28
5.1 @JsonProperty	28
5.2 @JsonFormat	28
5.3 @JsonInclude	28
5.4 @JsonIgnore	28
5.5 Las anotaciones son parte del contrato	28
Pregunta	29
Respuesta razonada	29
Resumen del Punto 1.3	29
Punto 1.4 - Diseño de APIs REST	29
Objetivos de aprendizaje	29
Bloque 1 - Qué es REST	30
1.1 REST es un estilo arquitectónico	30
1.2 Recursos y representaciones	30
1.3 Restricciones que dan sentido a REST	30
Pregunta	30
Respuesta razonada	30
Bloque 2 - Diseño de URLs	31
2.1 La URL identifica; el método actúa	31
2.2 Colecciones, elementos y subrecursos	31
2.3 Convenciones de nomenclatura	31
2.4 Versionado	31
Pregunta	31
Respuesta razonada	32
Bloque 3 - Métodos HTTP y códigos de estado	32
3.1 La semántica de los métodos	32
3.2 El código de estado forma parte del contrato	32
3.3 ResponseEntity cuando necesitamos controlar HTTP	32
Pregunta	33
Respuesta razonada	33
Bloque 4 - Paginación, filtrado y ordenación	33
4.1 Los query parameters modifican una consulta	33
4.2 Filtrado	33
4.3 Paginación	33
4.4 Ordenación	33
Pregunta	33
Respuesta razonada	34

Bloque 5 - Documentación y evolución	34
5.1 El contrato debe poder descubrirse	34
5.2 Documentar comportamiento, no sólo rutas	34
5.3 Evolución compatible	34
5.4 Diseño antes de código	34
Pregunta	34
Respuesta razonada	35
Resumen del punto 1.4	35
Punto 1.5 - Primer CRUD con DTOs	35
Objetivos de aprendizaje	35
Bloque 1 - El ciclo CRUD	35
1.1 Qué significa CRUD	35
1.2 Crear no es sólo devolver el cuerpo	36
1.3 Read: colección e individuo no significan lo mismo	36
Pregunta	36
Respuesta razonada	37
Bloque 2 - PUT en detalle	37
2.1 PUT reemplaza una representación completa	37
2.2 Idempotencia de PUT	37
2.3 PUT existente e inexistente	37
Pregunta	37
Respuesta razonada	37
Bloque 3 - PATCH y DELETE en detalle	38
3.1 PATCH modifica parcialmente	38
3.2 Idempotencia de PATCH: cuidado con las generalizaciones	38
3.3 DELETE elimina el recurso	38
Pregunta	38
Respuesta razonada	39
Bloque 4 - Códigos de estado en el CRUD	39
4.1 Tabla de contrato	39
4.2 400 no es lo mismo que 415	39
4.3 404 y 405 tampoco son equivalentes	39
4.4 Consistencia	39
Pregunta	39
Respuesta razonada	40
Bloque 5 - Consolidación del Módulo 1	40
5.1 Lo que ya sabemos construir	40
5.2 Lo que todavía no hemos resuelto	40
5.3 La generación de ID es intencionadamente simple	40
5.4 Filtro y ordenación siguen siendo consulta	41
5.5 El final de M1 es el punto de partida de M2	41
Pregunta	41
Respuesta razonada	41
Resumen del punto 1.5	41

Módulo 1 - Introducción a Spring Boot y arquitectura back-end

Propósito del módulo

En M0 construimos y verificamos el terreno: Java, Maven, Spring Initializr, la estructura del proyecto, el arranque, los logs, los tests y dos endpoints sencillos. En M1 empezamos a estudiar **por qué** esa aplicación funciona como lo hace y qué conceptos necesitamos para convertirla progresivamente en una API REST.

El módulo recorrerá cinco ideas encadenadas: primero entenderemos el papel de Spring Boot; después estudiaremos la comunicación cliente-servidor y HTTP; a continuación JSON y Jackson; después las

convenciones de diseño REST; y terminaremos con un primer CRUD en memoria basado en DTOs.

El proyecto seguirá siendo acumulativo. Cada punto debe poder explicarse sobre el código real que el alumno tiene delante, no sobre ejemplos desconectados.

Baseline del curso: Java 17, Maven 3.9.x mediante Maven Wrapper 3.9.16 y Spring Boot 3.5.16.

Punto 1.1 - Qué es Spring Boot y su papel en aplicaciones modernas

Objetivos de aprendizaje

Al finalizar este punto serás capaz de:

1. Explicar qué problema intenta resolver Spring Boot y qué trabajo de infraestructura reduce.
2. Distinguir con precisión Spring Framework de Spring Boot y entender por qué no son tecnologías rivales.
3. Explicar las tres ideas que dominan el modelo mental inicial de Spring Boot: **auto-configuración**, **starters** y **servidor embebido**.
4. Relacionar el `pom.xml`, el `classpath` y la configuración con decisiones automáticas que Spring Boot toma al arrancar.
5. Interpretar de forma básica el *Conditions Evaluation Report* y diferenciar coincidencias positivas, negativas y exclusiones.
6. Explicar por qué una configuración automática debe poder sobrescribirse.
7. Entender qué aporta un starter frente a declarar y coordinar manualmente un conjunto de dependencias.
8. Explicar por qué una aplicación Spring Boot web puede distribuirse como un JAR ejecutable que contiene su servidor.
9. Reconocer por qué este modelo encaja bien con APIs, servicios independientes y despliegues automatizados.
10. Separar una comodidad de desarrollo de una garantía real: Spring Boot reduce configuración repetitiva, pero no elimina la necesidad de comprender lo que ocurre.

Bloque 1 - Qué es Spring Boot y por qué existe

1.1 El problema no empieza en la lógica de negocio

Imagina que debes desarrollar un sistema que reciba peticiones HTTP, aplique reglas de negocio y devuelva datos. La lógica que realmente aporta valor quizá sea pequeña al principio: consultar un expediente, calcular una condición o devolver un saludo. Sin embargo, para que esa lógica sea accesible desde otra máquina necesitas infraestructura.

Como mínimo aparecen preguntas como estas:

- ¿qué proceso escuchará peticiones de red?;
- ¿qué servidor HTTP utilizaremos?;
- ¿cómo se transformará una URL en una llamada a un método Java?;
- ¿cómo se convertirán objetos Java en JSON?;
- ¿cómo leeremos configuración externa?;
- ¿cómo registraremos logs?;
- ¿cómo se coordinarán versiones compatibles de todas las librerías?;
- ¿cómo se empaquetará y arrancará la aplicación?;

Nada de eso es todavía la regla de negocio que queríamos programar, pero todo ello es necesario para que la aplicación exista como servicio.

Spring Boot nace precisamente para reducir ese coste de arranque y de configuración repetitiva. Es más exacto describirlo como una **capa opinada de arranque, integración y configuración sobre el ecosistema Spring** que como un sustituto de Spring Framework. Nos proporciona convenciones, auto-configuración, starters, herramientas de construcción y un modelo de ejecución que permiten obtener rápidamente una aplicación coherente.

La palabra *opinada* es importante. Spring Boot toma decisiones razonables por defecto. Eso no significa que todas las aplicaciones deban aceptar esas decisiones para siempre. Significa que empezamos desde una configuración funcional y cambiamos sólo lo que necesitamos cambiar.

1.2 Convención frente a configuración manual

Una aplicación configurable puede diseñarse de dos maneras extremas.

Enfoque A: todo explícito desde el primer minuto. El equipo elige cada implementación, crea cada objeto de infraestructura, declara cada dependencia y configura cada componente incluso cuando utiliza opciones estándar.

Enfoque B: convenciones sensatas con posibilidad de sustitución. El framework observa el contexto, aplica una configuración habitual y deja puntos claros para modificarla.

Spring Boot se acerca al segundo enfoque. Si encuentra una aplicación web basada en Spring MVC y las dependencias apropiadas, prepara gran parte de la infraestructura web. Si encuentra determinadas tecnologías de datos, seguridad, mensajería u observabilidad, puede activar configuraciones relacionadas. Si esas condiciones no existen, esas configuraciones no deben activarse.

La consecuencia práctica es que el tiempo inicial se dedica antes a comportamiento de aplicación y menos a repetir configuración conocida.

1.3 Lo que Spring Boot no hace por ti

La comodidad de Spring Boot puede producir una mala conclusión: “si arranca, ya no necesito entender nada”. Es justo al revés. Cuanto más trabajo realiza una plataforma por convención, más importante es saber **qué decisión ha tomado y por qué** cuando algo no funciona.

Spring Boot no decide por nosotros:

- las reglas del negocio;
- qué recursos debe exponer la API;
- qué datos son válidos;
- qué errores debe comunicar el sistema;
- qué límites de seguridad necesita;
- cómo modelar transacciones complejas;
- qué arquitectura es adecuada para el problema;
- qué observabilidad necesita producción.

Tampoco convierte una mala dependencia o una mala decisión de diseño en una buena decisión. Automatiza infraestructura conocida; no sustituye el razonamiento de ingeniería.

Pregunta

Si Spring Boot puede configurar muchas cosas automáticamente, ¿por qué merece la pena aprender Maven, el classpath, HTTP y los logs?

Respuesta razonada

Porque la automatización se apoya precisamente en esas piezas. La auto-configuración toma decisiones a partir de las clases disponibles, las propiedades y el contexto; Maven determina buena parte del classpath; HTTP define el contrato de comunicación; y los logs muestran qué ocurrió. Sin ese modelo mental sólo podemos probar cambios al azar. Con él podemos formular hipótesis y comprobarlas.

1.4 Las tres ideas clave para empezar

Durante este punto utilizaremos tres mecanismos como mapa mental:

1. **Auto-configuración.** Spring Boot evalúa el contexto y activa configuraciones cuando se cumplen determinadas condiciones.
2. **Starters.** Dependencias de conveniencia que reúnen un conjunto coherente de tecnologías para una capacidad concreta.
3. **Servidor embebido.** El servidor web forma parte de la aplicación ejecutable en lugar de ser necesariamente una instalación externa en la que desplegar un WAR.

Estas tres ideas no resumen todo Spring Boot, pero explican gran parte de la experiencia inicial del alumno.

Pregunta

¿Cuál de las tres ideas tiene más impacto en el día a día?

Respuesta razonada

No existe una respuesta única. Los starters reducen decisiones y errores de dependencias; la auto-configuración elimina gran cantidad de configuración repetitiva; y el servidor embebido simplifica ejecución y despliegue. Su verdadero valor aparece al combinarlas: declaramos una capacidad de alto nivel, Spring Boot reconoce el entorno resultante y la aplicación puede arrancar como un proceso autocontenido.

Bloque 2 - Spring Framework y Spring Boot

2.1 Spring Framework es la base

Spring Framework es el conjunto de tecnologías sobre el que se apoyan muchas de las capacidades que usaremos. Entre sus áreas más conocidas se encuentran:

- el contenedor de inversión de control y la inyección de dependencias;
- Spring MVC para aplicaciones web;
- soporte de acceso a datos y transacciones;
- integración con sistemas externos;
- testing del ecosistema Spring.

Spring Boot **no reemplaza** esas capacidades. Las utiliza y facilita su configuración, selección y arranque.

Un modo útil de recordarlo es:

Spring Framework -> capacidades y modelo de programación

Spring Boot -> arranque, convenciones, integración y configuración opinada

Cuando escribimos `@RestController`, `@GetMapping` o trabajamos con el contenedor, estamos utilizando conceptos del ecosistema Spring. Cuando una aplicación obtiene automáticamente un servidor, configuración MVC coherente y un conjunto gestionado de dependencias a partir de starters y condiciones, estamos viendo el valor añadido de Spring Boot.

2.2 Del Spring altamente configurado al arranque opinado

Históricamente, desarrollar con Spring podía exigir bastante configuración explícita. La configuración XML fue durante años habitual; posteriormente la configuración Java con `@Configuration` y `@Bean` redujo parte de esa carga, pero seguía siendo necesario decidir y conectar numerosas piezas.

Spring Boot cambia el punto de partida. En lugar de preguntar “¿cómo configuro desde cero todo lo necesario para una aplicación web?”, empezamos por “quiero una aplicación web” y declaramos el starter correspondiente. A partir de ahí Spring Boot intenta construir una configuración convencional y coherente.

No debemos caricaturizar la historia como “Spring Framework era malo y Spring Boot lo arregló”. Spring Framework aportó el modelo de componentes, inyección, MVC y muchas otras capacidades. Spring Boot apareció para reducir el coste de montar y operar combinaciones habituales de esas capacidades.

2.3 Separar capacidad de política de arranque

Esta separación ayuda a interpretar errores.

Si una anotación de Spring MVC no se comporta como esperas, quizá el problema esté en el modelo web o en el código del controlador.

Si una infraestructura que esperabas no se crea, quizá el problema esté en las **condiciones de auto-configuración**, en el classpath o en una propiedad.

Si una clase ni siquiera existe para el compilador, quizá el problema esté antes: Maven no ha resuelto la dependencia que la contiene.

Esta jerarquía evita tratar todos los fallos como “un problema de Spring Boot”.

Pregunta

¿Podemos utilizar Spring Framework sin Spring Boot?

Respuesta razonada

Sí. Spring Boot no es un requisito conceptual para usar Spring Framework. Podríamos configurar manualmente una aplicación basada en Spring. Spring Boot aporta un camino mucho más cómodo y homogéneo para la mayoría de aplicaciones modernas, pero la capacidad subyacente sigue perteneciendo al ecosistema Spring.

Pregunta

¿Podemos decir que Spring Boot “oculta” Spring Framework?

Respuesta razonada

Sólo parcialmente y como simplificación. Boot oculta mucha configuración repetitiva, pero el modelo de programación sigue siendo visible: beans, componentes, MVC, inyección, propiedades y demás conceptos no desaparecen. A medida que avancemos en el curso iremos trabajando cada vez más directamente con ellos.

Bloque 3 - Auto-configuración

3.1 Qué significa realmente “automática”

Auto-configurar no significa adivinar. Spring Boot dispone de configuraciones preparadas que se activan o no según condiciones observables. Entre las señales que pueden intervenir están:

- clases presentes o ausentes en el classpath;
- beans ya definidos por la aplicación;
- propiedades de configuración;
- tipo de aplicación;
- recursos disponibles;
- otras configuraciones activas.

La idea puede expresarse así:

```
si se cumplen ciertas condiciones
-> aplica una configuración convencional
```

si la aplicación ya aporta una alternativa o falta un requisito
-> no aplica esa configuración

Esto explica por qué modificar una dependencia puede alterar el comportamiento del arranque incluso sin cambiar una línea de Java: ha cambiado el conjunto de condiciones que Spring Boot observa.

3.2 @SpringBootApplication y la auto-configuración

En nuestro proyecto la clase principal contiene:

```
@SpringBootApplication
public class MiProyectoApplication {
    public static void main(String[] args) {
        SpringApplication.run(MiProyectoApplication.class, args);
    }
}
```

En M0 vimos @SpringBootApplication como una anotación compuesta que reúne configuración, auto-configuración y escaneo de componentes. En M1 damos un paso más: su parte de auto-configuración habilita el mecanismo por el que Spring Boot evalúa configuraciones candidatas durante el arranque.

No significa “configura todo”. Significa “considera el conjunto de auto-configuraciones disponibles y aplica las que correspondan”.

3.3 Condiciones positivas y negativas

Al arrancar con --debug, Spring Boot puede imprimir un **Conditions Evaluation Report**.

Ese informe permite observar tres ideas:

Positive matches. Configuraciones cuyas condiciones se han cumplido.

Negative matches. Configuraciones candidatas que no se han aplicado porque alguna condición no se cumple.

Exclusions. Configuraciones que han sido excluidas expresamente.

Una coincidencia negativa no es necesariamente un error. Si nuestra aplicación no utiliza JMS, es normal que una configuración que depende de clases JMS no se active. El informe es una explicación del proceso de decisión, no una lista de fallos.

3.4 Ejemplo conceptual: aplicación web

Nuestro pom.xml declara spring-boot-starter-web. Ese starter hace que aparezcan en el classpath piezas relacionadas con Spring Web y Tomcat. Durante el arranque, las condiciones relevantes detectan esas clases y el contexto de aplicación, y Spring Boot puede preparar la infraestructura MVC y el servidor web.

Observa la cadena causal:

```
pom.xml
-> Maven resuelve starter y transitivas
-> cambia el classpath
-> las condiciones ven determinadas clases
-> se activan auto-configuraciones
-> aparecen beans e infraestructura
-> la aplicación atiende HTTP
```

Esta cadena conecta algo que en un IDE parece simplemente “añadir una dependencia” con una consecuencia real en runtime.

3.5 Back-off: la configuración automática debe ceder

Una característica esencial de una auto-configuración útil es que no debe convertirse en una cárcel. Cuando la aplicación define explícitamente una pieza que sustituye al valor por defecto, muchas auto-configuraciones están diseñadas para **retroceder** (*back off*) y respetar la decisión del usuario.

También podemos sobrescribir numerosas propiedades. Por ejemplo, el servidor escucha en 8080 por defecto, pero:

```
server.port=9090
```

cambia ese comportamiento sin modificar el código Java.

Esto ilustra una regla importante:

El valor por defecto sirve para arrancar rápido; la configuración explícita sirve para adaptar el sistema.

Pregunta

¿Por qué es importante que una auto-configuración pueda sobrescribirse o ceder ante una configuración explícita?

Respuesta razonada

Porque un sistema que sólo funciona mientras aceptemos todas sus decisiones por defecto no es realmente configurable. Las convenciones son valiosas cuando reducen trabajo repetitivo, pero una aplicación real necesita adaptar puertos, seguridad, serialización, conexiones, cachés y muchas otras piezas. El objetivo es evitar configuración innecesaria, no impedir configuración necesaria.

3.6 Diagnóstico: no memorizar cientos de auto-configuraciones

No necesitas memorizar todas las clases de auto-configuración. Necesitas dominar una estrategia:

1. identifica qué infraestructura esperabas;
2. comprueba que la dependencia necesaria está en el classpath;
3. comprueba propiedades relevantes;
4. arranca con `--debug` cuando necesites explicación adicional;
5. busca la configuración relacionada;
6. lee por qué hizo *match* o por qué no lo hizo.

Eso convierte un sistema aparentemente “mágico” en un sistema observable.

Bloque 4 - Starters y gestión coherente de dependencias

4.1 Qué es un starter

Un starter es una dependencia de conveniencia diseñada para expresar una capacidad de alto nivel. En lugar de obligar al alumno a seleccionar manualmente todas las librerías necesarias para construir una aplicación web, podemos declarar:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

A partir de ahí Maven resuelve las dependencias transitivas que ese starter necesita.

El starter no debe imaginarse como “una megabiblioteca que contiene físicamente todo”. Su valor principal es **declarativo**: expresa un conjunto coherente de dependencias para una finalidad.

4.2 Dependencias directas y transitivas

En nuestro POM declaramos `spring-boot-starter-web` directamente. Otras librerías aparecen porque son dependencias de ese starter o de sus dependencias.

Podemos comprobarlo con:

```
./mvnw dependency:tree
```

En Windows:

```
mvnw.cmd dependency:tree
```

Ahí veremos piezas como Spring Web MVC, Jackson y Tomcat dentro del árbol resuelto. La versión exacta de cada artefacto no debe memorizarse; lo importante es entender de dónde procede y quién gestiona su compatibilidad.

4.3 Starter no significa “usar todo en cada petición”

Que una librería esté disponible no significa que cada endpoint la utilice siempre. Nuestro `/hola` devuelve texto; Jackson puede estar presente en el classpath, pero ese `String` sencillo no demuestra por sí solo una serialización compleja de DTO.

Este matiz será importante en la práctica: para demostrar el efecto de Jackson necesitaremos un endpoint temporal que obligue realmente a producir JSON estructurado.

4.4 Versiones coordinadas

Spring Boot proporciona gestión de dependencias para un conjunto amplio de librerías compatibles. Por eso el POM de nuestro proyecto no necesita escribir una versión manual junto a cada starter.

La ventaja no es sólo escribir menos XML. El beneficio mayor es reducir combinaciones arbitrarias de versiones que podrían ser incompatibles.

Eso no significa que una actualización sea siempre trivial. Cambiar de línea de Spring Boot puede implicar cambios de API o comportamiento. Significa que partimos de un conjunto coordinado en lugar de construir uno al azar.

Pregunta

¿Qué problema evita coordinar versiones mediante Spring Boot frente a declarar manualmente una versión independiente para cada librería?

Respuesta razonada

Reduce el riesgo de construir un classpath incoherente. Si elegimos por separado versiones de Spring MVC, Jackson, Tomcat y decenas de dependencias, podemos seleccionar combinaciones que nunca se probaron juntas. La gestión centralizada proporciona un conjunto conocido y permite que el equipo razone sobre una versión de plataforma en lugar de docenas de decisiones aisladas.

4.5 Starter frente a dependencia específica

Los starters son excelentes para capacidades amplias, pero no toda dependencia de un proyecto debe ser un starter. A lo largo del curso añadiremos también librerías concretas cuando exista una razón explícita.

La regla pedagógica es comprender **qué capacidad estamos pidiendo** y **qué dependencia la introduce**, no acumular starters preventivamente.

Bloque 5 - Servidor embebido y aplicaciones modernas

5.1 Dos modelos de despliegue

En el modelo tradicional de muchas aplicaciones Java web se construía un WAR y se desplegaba en un servidor de aplicaciones o contenedor servlet instalado y administrado por separado.

Esquema simplificado:

```
servidor instalado
  |-- Tomcat
  |-- despliegue de aplicacion.war
```

Con el modelo habitual de Spring Boot para una aplicación web, el servidor puede viajar como parte de la aplicación ejecutable:

```
java -jar mi-proyecto.jar
-> arranca la JVM
-> arranca Spring Boot
-> arranca Tomcat embebido
-> publica el puerto HTTP
```

En nuestro curso utilizamos este segundo modelo.

5.2 “Embebido” no significa que Tomcat deje de existir

Tomcat sigue siendo un servidor web/servlet real. La diferencia es cómo se empaqueta y quién controla su ciclo de vida.

No necesitamos instalar un Tomcat externo para ejecutar el proyecto. Maven resuelve las librerías de Tomcat y Spring Boot crea y arranca el servidor apropiado cuando la aplicación web se inicia.

Por tanto, el experimento correcto no es sólo preguntar “¿tengo instalado Tomcat?”. La evidencia fuerte consiste en demostrar simultáneamente que:

- Tomcat está en el árbol de dependencias de la aplicación;
- el JAR puede arrancar por sí mismo con Java;
- el proceso publica el puerto HTTP;
- no depende de desplegarse manualmente en una instalación externa.

5.3 Ventajas del modelo embebido

Despliegue sencillo. El artefacto ejecutable concentra aplicación y runtime web necesario.

Reproducibilidad. El proyecto controla qué versión del servidor entra en su conjunto de dependencias.

Aislamiento. Dos aplicaciones pueden llevar ciclos de versión distintos sin compartir obligatoriamente una única instalación global de Tomcat.

Automatización. Un proceso que se inicia con un comando bien definido encaja naturalmente en pipelines, servicios del sistema y contenedores.

Desarrollo coherente. El mismo modelo básico de arranque se utiliza en la máquina del alumno y en otros entornos, aunque configuración y operación de producción sean mucho más estrictas.

5.4 Costes y límites

El servidor embebido no elimina los problemas operativos. Seguimos necesitando:

- controlar memoria y CPU;
- configurar puertos y TLS cuando corresponda;
- gestionar logs;
- monitorizar disponibilidad;
- aplicar actualizaciones de seguridad;

- planificar despliegues y rollback;
- configurar proxies o balanceadores cuando la arquitectura los necesite.

Además, un JAR ejecutable incluye numerosas dependencias y por eso pesa más que un artefacto que presuponga un servidor ya instalado.

5.5 APIs, servicios y contenedores

El modelo “una aplicación = un proceso ejecutable con su servidor” encaja bien con arquitecturas de servicios y con contenedores. No porque Docker o Kubernetes exijan Spring Boot, sino porque es sencillo empaquetar y automatizar una aplicación que ya sabe arrancarse a sí misma.

Un contenedor no sustituye al servidor embebido: normalmente contiene el proceso Java que, a su vez, arranca Spring Boot y su servidor web.

5.6 Monolito y microservicio no son sinónimos de viejo y moderno

Spring Boot puede utilizarse para aplicaciones monolíticas, servicios pequeños, APIs internas y otros estilos. Elegir microservicios no es una consecuencia automática de usar Spring Boot.

La tecnología facilita crear procesos independientes, pero la decisión arquitectónica debe responder a necesidades de dominio, despliegue, escalado, organización y operación. Dividir un sistema sin necesidad puede aumentar complejidad de red, datos, observabilidad y despliegue.

Pregunta

¿Por qué el servidor embebido facilita el despliegue sin convertirlo automáticamente en “producción lista”?

Respuesta razonada

Porque reduce una parte concreta del problema: empaquetar y arrancar la infraestructura web junto con la aplicación. Producción incluye además seguridad, configuración, secretos, monitorización, escalado, backups, redes, actualizaciones y procedimientos operativos. Un JAR ejecutable simplifica el vehículo de despliegue, no toda la explotación del sistema.

Cómo se combinan las tres ideas en nuestro proyecto

Nuestro proyecto M1 parte del snapshot final de M0. Si seguimos la cadena completa:

1. pom.xml declara spring-boot-starter-web.
2. Maven resuelve el starter y sus dependencias transitivas.
3. El classpath contiene Spring MVC, Jackson y Tomcat, entre otras piezas.
4. @SpringBootApplication habilita el modelo de configuración y auto-configuración.
5. Spring Boot evalúa condiciones durante el arranque.
6. Se crea infraestructura web apropiada.
7. Tomcat embebido escucha en el puerto configurado.
8. Spring MVC descubre SaludoController mediante component scan.
9. GET /hola se asigna al método saludar().
10. El navegador o curl recibe la respuesta.

En M0 observamos partes de esta cadena. En 1.1 el objetivo es **explicarla y demostrarla con evidencias**.

Resumen del Punto 1.1

- Spring Boot reduce configuración repetitiva mediante convenciones y auto-configuración.
- Spring Boot no sustituye Spring Framework; se apoya en él y facilita su integración y arranque.
- La auto-configuración evalúa condiciones; no “adivina” y no configura todo indiscriminadamente.

- El *Conditions Evaluation Report* permite observar por qué ciertas configuraciones se aplicaron o no.
- Una buena auto-configuración admite sobrescritura y *back-off*.
- Los starters expresan capacidades de alto nivel y arrastran dependencias transitivas coherentes.
- Maven y el classpath forman parte del mecanismo: cambiar dependencias puede cambiar las condiciones de runtime.
- El servidor embebido viaja con la aplicación y permite arrancarla como un proceso Java autocontenido.
- Este modelo encaja bien con APIs y despliegues automatizados, pero no decide por sí solo la arquitectura ni resuelve la operación de producción.
- Comprender estas piezas convierte la “magia de Spring Boot” en un sistema que podemos inspeccionar, verificar y diagnosticar.

El siguiente punto estudiará la arquitectura cliente-servidor y HTTP. Antes de diseñar APIs necesitamos comprender qué viaja entre cliente y servidor, cómo se representa una petición y por qué 200, 404 o 405 describen situaciones diferentes.

Punto 1.2 - Arquitectura cliente-servidor y HTTP

Objetivos de aprendizaje

Al finalizar este punto serás capaz de:

1. Explicar con precisión qué papel desempeñan cliente y servidor en una comunicación de red.
2. Describir el ciclo petición-respuesta desde que el usuario introduce una URL hasta que recibe una representación.
3. Diferenciar front-end y back-end sin confundir esa separación con una tecnología concreta.
4. Identificar los componentes esenciales de una petición HTTP: método, destino, cabeceras y, cuando existe, cuerpo.
5. Identificar los componentes esenciales de una respuesta HTTP: estado, cabeceras y cuerpo.
6. Distinguir los métodos HTTP principales por su intención semántica y no sólo por su nombre.
7. Interpretar los códigos de estado más habituales como parte del contrato entre cliente y servidor.
8. Utilizar navegador, DevTools y curl para observar tráfico real en lugar de razonar únicamente desde el código Java.
9. Diferenciar errores de conexión, rutas inexistentes, métodos no permitidos, contenido no interpretable y fallos internos.
10. Relacionar HTTP con el proyecto Spring Boot que ya tienes funcionando.

Bloque 1 - Cliente y servidor

1.1 Dos papeles, no necesariamente dos tipos de máquina

En una arquitectura cliente-servidor, **cliente** y **servidor** describen roles dentro de una interacción.

El cliente inicia una petición. El servidor permanece preparado para recibir peticiones, procesarlas y producir respuestas.

En nuestro ejercicio:

```
curl o navegador -> cliente
mi-proyecto      -> servidor HTTP
```

Pero los roles no están ligados para siempre a un programa. Una aplicación Spring Boot puede actuar como servidor frente a un navegador y, unos milisegundos después, actuar como cliente de otro servicio remoto.

Esto evita una simplificación frecuente: “el cliente es el ordenador del usuario y el servidor es una máquina grande en un centro de datos”. En la práctica, ambos roles pueden ejecutarse incluso en la misma máquina, como ocurre cuando accedemos a `localhost:8080` durante el curso.

1.2 localhost, dirección y puerto

Cuando utilizamos:

`http://localhost:8080/hola`

estamos proporcionando varias piezas de información:

- `http`: protocolo/esquema que indica cómo queremos comunicarnos;
- `localhost`: nombre que identifica la propia máquina;
- `8080`: puerto TCP en el que espera el servidor;
- `/hola`: ruta del recurso o endpoint que queremos invocar.

En condiciones habituales `localhost` se resuelve hacia una dirección de loopback como `127.0.0.1` para IPv4 o `::1` para IPv6. Conviene no memorizar que **siempre** será exactamente `127.0.0.1`: el detalle puede variar según sistema y resolución local. Lo importante es que el tráfico vuelve a la propia máquina.

El puerto permite que distintos servicios compartan una misma dirección IP. Un proceso puede escuchar en 8080 y otro en 9090 sin que sean el mismo servidor lógico.

1.3 Una URL no identifica un método Java

El cliente conoce una URL y un método HTTP. No sabe que en el servidor existe una clase `SaludoController` ni un método `saludar()`.

Esa traducción pertenece al back-end:

```
GET /hola
  -> infraestructura HTTP
  -> Spring MVC
  -> mapping compatible
  -> método Java
```

Esta separación es fundamental. Permite modificar internamente clases y métodos sin obligar a un cliente a conocer la implementación, siempre que el contrato HTTP externo se conserve.

Pregunta

Si navegador y servidor se ejecutan en el mismo PC, ¿sigue existiendo una arquitectura cliente-servidor?

Respuesta razonada

Sí. La arquitectura describe responsabilidades y el flujo de comunicación, no la distancia física. El navegador sigue iniciando una petición y la aplicación Spring Boot sigue escuchando, procesando y respondiendo. `localhost` simplemente hace que ambos procesos estén en la misma máquina.

Bloque 2 - El ciclo petición-respuesta

2.1 HTTP organiza una conversación

HTTP sigue un modelo petición-respuesta. Simplificando:

```
CLIENTE                                SERVIDOR
|                                     |
| ----- petición -----> |
|                                     | procesa
```

```
| <----- respuesta ----- |
|                               |
```

El servidor no “manda una página porque sí” al navegador en este modelo básico. Responde a una petición concreta.

2.2 Qué contiene una petición

Una petición HTTP tiene una línea inicial, cabeceras y, dependiendo del método y del caso de uso, un cuerpo.

Ejemplo conceptual:

```
GET /hola HTTP/1.1
Host: localhost:8080
Accept: */*
User-Agent: curl/8.x
```

Para un POST con datos podemos encontrar además:

```
POST /recurso HTTP/1.1
Host: localhost:8080
Content-Type: application/json
Content-Length: ...
```

```
{"campo": "valor"}
```

Las partes cumplen funciones distintas:

- **método:** expresa la intención de la operación;
- **target/ruta:** identifica dónde se aplica;
- **versión HTTP:** indica el protocolo utilizado en la conversación;
- **cabeceras:** aportan metadatos;
- **cuerpo:** transporta una representación cuando es necesario.

2.3 Qué contiene una respuesta

Ejemplo simplificado:

```
HTTP/1.1 200 OK
Content-Type: text/plain;charset=UTF-8
Content-Length: ...
```

Hola, Ministerio de Educación

De nuevo distinguimos:

- **código de estado:** resultado general de la operación;
- **cabeceras:** metadatos de la respuesta;
- **cuerpo:** representación devuelta al cliente cuando existe.

El código y el cuerpo no son intercambiables. Un texto que diga “todo correcto” dentro de una respuesta 500 sigue siendo, para HTTP, un error de servidor. Del mismo modo, devolver 200 con un texto “no encontrado” es un contrato pobre si la situación debería expresarse como 404.

2.4 Del socket al controlador

Para nuestro GET /hola, el recorrido observable puede simplificarse así:

1. el cliente resuelve el destino;
2. abre una conexión hacia el puerto 8080;
3. envía una petición HTTP;
4. Tomcat embebido recibe bytes de red y los interpreta según el protocolo;

5. la infraestructura servlet/Spring MVC procesa la petición;
6. el DispatcherServlet coordina la búsqueda de un handler;
7. el mapping encuentra GET /hola;
8. se ejecuta SaludoController.saludar();
9. Spring construye la respuesta;
10. Tomcat la envía al cliente.

En este punto no necesitamos estudiar todos los componentes internos de Spring MVC. Sí necesitamos entender dónde termina HTTP y dónde empieza nuestro código.

Pregunta

¿Por qué una petición y una respuesta tienen cabeceras además del cuerpo?

Respuesta razonada

Porque el cuerpo transporta principalmente la representación o datos, mientras que las cabeceras describen cómo interpretar o gestionar la comunicación: tipo de contenido, longitud, capacidades aceptadas, autenticación, caché y muchos otros metadatos. Separar datos de metadatos evita mezclar contrato de transporte con contenido de negocio.

Bloque 3 - Front-end y back-end

3.1 Responsabilidades diferentes

Front-end es la parte orientada a la interacción con el usuario o consumidor visual de una aplicación: interfaz, navegación, formularios, representación de datos y lógica de presentación.

Back-end es la parte que recibe peticiones, aplica reglas, coordina servicios y datos y produce respuestas consumibles por clientes.

En una aplicación web moderna típica:

```
front-end
  |
  | HTTP/JSON
  v
back-end Spring Boot
  |
  v
servicios / datos / otros sistemas
```

La separación no obliga a utilizar JavaScript en el front ni Java en el back. Es una separación de responsabilidades y contratos.

3.2 Por qué separar

Separar front y back puede aportar:

- evolución independiente de la interfaz y la lógica del servidor;
- reutilización del mismo back-end por web, móvil u otros sistemas;
- contratos HTTP más claros;
- especialización de responsabilidades;
- posibilidad de desplegar o escalar componentes de forma diferente cuando sea necesario.

También añade costes: coordinación de contratos, errores de red, CORS en ciertos escenarios, versionado, observabilidad distribuida y más puntos donde algo puede fallar. No debemos presentar la separación como gratuita.

3.3 API como frontera

Una API HTTP define una frontera. El cliente debería depender de aspectos como:

- URL;
- método;
- cabeceras relevantes;
- forma del cuerpo;
- códigos de estado.

No debería depender de nombres privados de clases, campos que no forman parte del contrato o detalles internos de persistencia.

Más adelante formalizaremos esa frontera como API REST. En 1.2 nos basta comprender la conversación HTTP sobre la que REST se apoya.

Pregunta

¿Separar front-end y back-end significa que deben estar siempre en repositorios o servidores distintos?

Respuesta razonada

No. Pueden estar separados conceptualmente y compartir repositorio o despliegue en ciertos proyectos. La separación importante aquí es de responsabilidades y del contrato de comunicación. Las decisiones de repositorio, red y despliegue son adicionales.

Bloque 4 - Métodos HTTP y códigos de estado

4.1 El método forma parte del endpoint

Una ruta por sí sola no describe toda la operación. Para Spring MVC no es equivalente:

```
GET /hola
POST /hola
```

Nuestro controlador declara `@GetMapping("/hola")`. Por eso `GET /hola` puede resolverse, mientras que un `POST /hola` no encuentra un handler que acepte esa combinación y produce normalmente `405 Method Not Allowed`.

Este detalle prepara una idea central de REST: el método comunica intención.

4.2 Métodos principales

GET solicita una representación. En condiciones normales se utiliza para consultar y no debería diseñarse para provocar cambios de negocio por el mero hecho de leer.

POST envía una representación para que el servidor la procese; frecuentemente se usa para crear recursos, aunque HTTP no lo limita exclusivamente a creación.

PUT suele expresar reemplazo completo del estado de un recurso identificado. Más adelante estudiaremos su relación con idempotencia.

PATCH expresa una modificación parcial.

DELETE solicita eliminar un recurso o dejarlo no disponible según el contrato de la API.

En este punto sólo tenemos implementado GET. Los demás métodos se estudian para poder interpretar respuestas y diseñar después el CRUD.

4.3 Familias de códigos de estado

Los códigos HTTP se agrupan por familias:

- 1xx: información/progreso del protocolo;

- 2xx: la petición se procesó satisfactoriamente según su semántica;
- 3xx: redirección o necesidad de otra localización/acción;
- 4xx: la petición no puede atenderse por una condición atribuible al lado cliente o al recurso solicitado;
- 5xx: el servidor no pudo completar una petición que, desde la perspectiva del protocolo, llegó a él.

No debemos reducir 4xx a “el usuario se equivocó” ni 5xx a “Java lanzó una excepción”. Son categorías de respuesta HTTP.

4.4 Códigos que usaremos pronto

200 OK. Operación procesada correctamente con una respuesta normal.

201 Created. Se ha creado un recurso. Lo utilizaremos con POST cuando construyamos la API de alumnos.

204 No Content. La operación se completó y no hay cuerpo que devolver; es común en ciertos DELETE o actualizaciones.

400 Bad Request. El servidor no puede procesar la petición por su forma o contenido sintáctico/semántico básico. En la práctica provocaremos uno con JSON mal formado.

404 Not Found. No existe un recurso/ruta que pueda atender esa petición en el contexto dado.

405 Method Not Allowed. La ruta corresponde a una operación conocida, pero el método HTTP enviado no está permitido para ella.

415 Unsupported Media Type. El servidor no admite el tipo de representación indicado para la operación.

500 Internal Server Error. El servidor encontró un fallo inesperado al procesar la petición.

4.5 404 no es 405

Esta distinción será uno de los experimentos del punto:

```
GET /no-existe -> 404
POST /hola -> 405
```

En el primer caso no hay mapping para ese destino. En el segundo conocemos /hola, pero no con POST.

El cliente puede reaccionar de forma diferente porque las causas contractuales son distintas.

Pregunta

¿Por qué no devolver siempre 200 y explicar cualquier problema en el cuerpo?

Respuesta razonada

Porque destruiríamos una parte esencial del protocolo. Navegadores, clientes HTTP, proxies, herramientas de monitorización y nuestro propio código pueden tomar decisiones a partir del estado sin interpretar primero un cuerpo específico. Los códigos de estado proporcionan un vocabulario común y predecible.

Bloque 5 - Herramientas para observar HTTP

5.1 El navegador: útil, pero interpreta por ti

El navegador es excelente para realizar una petición GET de forma inmediata. Sin embargo, además de enviar HTTP, interpreta respuestas, renderiza contenido, sigue ciertas redirecciones y puede ocultar detalles detrás de una interfaz visual.

Por eso no debemos utilizar únicamente “lo que veo en la página” como evidencia de una API.

5.2 DevTools: el navegador se vuelve observable

La pestaña **Network** de las herramientas de desarrollador permite inspeccionar:

- URL y método;
- estado;
- cabeceras de petición;
- cabeceras de respuesta;
- cuerpo/response;
- tiempos de distintas fases.

Esto conecta la experiencia visual con la conversación HTTP real.

5.3 curl: cliente mínimo y reproducible

curl es muy útil para aprendizaje y diagnóstico porque permite describir una petición en un comando reproducible.

Sólo cuerpo:

```
curl http://localhost:8080/hola
```

Cabeceras de respuesta más cuerpo:

```
curl -i http://localhost:8080/hola
```

Conversación detallada:

```
curl -v http://localhost:8080/hola
```

Método explícito:

```
curl -i -X POST http://localhost:8080/hola
```

Cabecera y cuerpo:

```
curl -i -X POST http://localhost:8080/eco \
-H "Content-Type: application/json" \
-d '{"mensaje": "hola"}'
```

5.4 La salida exacta puede cambiar sin cambiar el concepto

Fechas, Content-Length, versión concreta de curl, ciertas cabeceras y el formato del cuerpo de error pueden variar según versión y configuración.

Por eso una guía técnica robusta separa:

Contrato estable que queremos verificar

estado 404

Content-Type coherente cuando exista cuerpo

ausencia de handler para la ruta

De una captura ilustrativa cuya literalidad puede cambiar:

Date: ...

Content-Length: ...

JSON exacto de error ...

La práctica te pedirá observar la respuesta real de tu versión antes de sacar conclusiones.

5.5 Diagnóstico por capas

Ante un fallo HTTP conviene preguntar en este orden:

1. **¿Hay proceso escuchando?** Si no, veremos errores de conexión antes de cualquier código HTTP.
2. **¿Llegamos al puerto correcto?** Un puerto equivocado puede llevar a otro proceso o a ningún proceso.
3. **¿Existe la ruta?** Si no, esperamos 404.
4. **¿El método es compatible?** Si no, esperamos 405.
5. **¿El tipo de contenido y el cuerpo son procesables?** Aquí aparecen 400/415 y errores de conversión.
6. **¿Falló el servidor mientras ejecutaba la operación?** Aquí puede aparecer un 5xx y debemos leer logs.

Esta secuencia evita interpretar `Connection refused` como si fuera “un 404 de Spring”. Si no hay conexión, HTTP ni siquiera llegó a empezar.

Pregunta

¿Qué aporta `curl -v` que no aporta simplemente abrir `/hola` en el navegador?

Respuesta razonada

Hace visible la conversación: conexión, línea de petición, cabeceras enviadas, estado y cabeceras recibidas. El navegador también usa HTTP, pero su interfaz normal oculta muchos detalles. DevTools y `curl -v` convierten esos detalles en evidencia observable.

Resumen del Punto 1.2

- Cliente y servidor son roles dentro de una comunicación, no tipos fijos de máquina.
- `localhost:8080` combina destino local y puerto; `/hola` identifica la ruta solicitada.
- HTTP organiza una conversación de petición y respuesta.
- Método, ruta, cabeceras y cuerpo describen la petición; estado, cabeceras y cuerpo describen la respuesta.
- Front-end y back-end separan responsabilidades y se coordinan mediante un contrato.
- El método HTTP forma parte de la operación: `GET /hola` y `POST /hola` no son equivalentes.
- Los códigos de estado comunican resultado de forma estándar; 404 y 405 describen problemas distintos.
- Navegador, DevTools y `curl` ofrecen perspectivas complementarias.
- Una salida concreta puede cambiar entre versiones; el contrato que verificamos debe distinguirse de detalles ilustrativos inestables.
- El diagnóstico debe comenzar en la capa más baja que todavía podría explicar el síntoma: conexión, puerto, ruta, método, representación y finalmente lógica del servidor.

En el punto 1.3 utilizaremos esta base HTTP para estudiar el formato que dominará los cuerpos de nuestras APIs: JSON, y el papel de Jackson al convertir entre representaciones JSON y objetos Java.

Punto 1.3 - JSON y Jackson

Objetivos de aprendizaje

Al finalizar este punto serás capaz de:

1. Explicar qué es JSON y por qué se utiliza de forma dominante en APIs HTTP.
2. Leer y escribir objetos y arrays JSON respetando sus tipos y reglas de sintaxis.

3. Distinguir JSON válido de texto que “se parece” a JSON pero contiene errores.
4. Aplicar convenciones de nombres, fechas, valores nulos y estructuras anidadas en una API.
5. Explicar la diferencia entre serialización y deserialización.
6. Describir cómo Spring MVC utiliza Jackson a través de sus conversores HTTP.
7. Entender qué papel tienen constructor, propiedades y accesores en la creación de DTOs deserializables.
8. Utilizar `@JsonProperty`, `@JsonFormat`, `@JsonInclude` y `@JsonIgnore` con una intención concreta.
9. Distinguir una decisión de contrato JSON de una decisión interna de nombres Java.
10. Diagnosticar errores de parseo, tipos incompatibles, fechas inválidas y propiedades desconocidas.

Bloque 1 - Qué es JSON

1.1 Un formato de intercambio de datos

JSON significa **JavaScript Object Notation**. Su sintaxis nació inspirada en la notación de objetos de JavaScript, pero JSON es un formato de datos independiente del lenguaje. Java, JavaScript, Python, C#, Go y prácticamente cualquier plataforma actual pueden producirlo y consumirlo.

Un ejemplo:

```
{
  "id": "12345",
  "titular": "Ana García López",
  "estado": "EN_TRAMITE",
  "importe": 1500.0,
  "activo": true
}
```

No es una clase Java ni un objeto vivo en memoria. Es una **representación textual estructurada**. Cuando viaja por HTTP, el cuerpo contiene bytes que representan ese texto y la cabecera suele declarar:

Content-Type: application/json

1.2 Por qué encaja bien en APIs

JSON se popularizó como alternativa más ligera a formatos más verbosos para intercambio de datos y hoy es el formato predominante de muchas APIs. Sus ventajas prácticas incluyen:

- sintaxis relativamente compacta;
- lectura razonable por humanos;
- soporte universal;
- representación natural de objetos, colecciones y estructuras anidadas;
- buena integración con navegadores y herramientas HTTP.

Esto no significa que sea el único formato posible. HTTP puede transportar texto, HTML, imágenes, XML, binario y muchos otros tipos. Una API elige JSON porque su contrato lo establece, no porque HTTP obligue a usarlo.

1.3 JSON frente a un objeto Java

Considera esta clase conceptual:

```
class ExpedienteDTO {
    private String identificador;
    private String titular;
    private Double importe;
}
```

Y esta representación:

```
{
  "id": "12345",
  "titular": "Ana García López",
  "importe": 1500.0
}
```

No hay una correspondencia automática impuesta por JSON entre identificador e id. Esa correspondencia la configura la capa de serialización. En nuestro caso la estableceremos con Jackson.

1.4 JSON no transporta comportamiento

Un objeto Java puede tener métodos, invariantes y lógica. JSON sólo expresa datos. Cuando serializamos un DTO no enviamos su bytecode ni sus métodos al cliente.

Esta diferencia es especialmente importante para una API: el cliente recibe un contrato de datos, no una copia ejecutable de nuestras clases internas.

Pregunta

¿Por qué decimos que JSON es independiente del lenguaje si su nombre contiene “JavaScript”?

Respuesta razonada

Porque el nombre describe su origen sintáctico, no una dependencia de ejecución. Un documento JSON es texto conforme a una gramática. Cualquier programa capaz de interpretar esa gramática puede leerlo o producirlo sin ejecutar JavaScript.

Bloque 2 - Sintaxis de JSON

2.1 Objetos y pares clave-valor

Un objeto JSON se delimita con llaves:

```
{
  "nombre": "Ana",
  "edad": 16
}
```

Las claves son cadenas entre comillas dobles. Cada clave se separa de su valor mediante : y los pares se separan mediante comas.

No es JSON válido escribir:

```
{nombre: 'Ana'}
```

porque la clave no tiene comillas dobles y la cadena utiliza comillas simples.

2.2 Arrays

Un array se delimita con corchetes:

```
["DNI.pdf", "Notas.pdf"]
```

Puede contener objetos:

```
[
  {"id": "1", "nombre": "Ana"},
  {"id": "2", "nombre": "Luis"}
]
```

O combinarse dentro de un objeto:

```
{  
  "id": "12345",  
  "documentos": ["DNI.pdf", "Notas.pdf"]  
}
```

2.3 Tipos de valor

JSON dispone de un conjunto pequeño de tipos:

- cadena;
- número;
- booleano true/false;
- null;
- objeto;
- array.

No existe un tipo nativo “fecha”. Por eso una fecha se representa mediante una convención, normalmente una cadena como:

"2025-01-15"

El acuerdo sobre esa cadena forma parte del contrato de la API.

2.4 Errores frecuentes

Estos ejemplos son inválidos:

Coma final:

```
{"mensaje": "hola", }
```

Comillas simples:

```
{'mensaje': 'hola'}
```

Clave sin comillas:

```
{mensaje: "hola"}
```

Llave sin cerrar:

```
{"mensaje": "hola"
```

Cuando `@RequestBody` necesita convertir uno de estos cuerpos a un objeto Java, el parser no puede construir una estructura válida y Spring MVC termina normalmente respondiendo 400 Bad Request.

2.5 JSON no admite comentarios estándar

Un documento JSON no dispone de comentarios `//` o `/* ... */` como Java. La documentación del significado de los campos debe vivir en documentación de API, esquemas, OpenAPI, ejemplos y nombres bien elegidos, no incrustada como comentarios dentro del payload.

Pregunta

¿Por qué una coma final que muchos lenguajes toleran en ciertas estructuras puede romper un JSON?

Respuesta razonada

Porque JSON tiene su propia gramática. No heredamos automáticamente todas las extensiones que un lenguaje de programación acepte. El parser valida la representación contra las reglas JSON, y una coma que anuncia otro elemento sin proporcionarlo deja la estructura incompleta.

Bloque 3 - Convenciones de JSON en APIs REST

3.1 Nombres predecibles

Una API debería adoptar una convención estable para nombres de propiedades. Usaremos camelCase:

```
{
  "fechaSolicitud": "2025-01-15",
  "nombreCompleto": "Ana García López"
}
```

Lo importante no es afirmar que sólo exista una convención válida, sino **ser coherentes**. Un cliente no debería encontrar `fechaSolicitud`, `nombre_completo` y `tipo-beca` mezclados sin motivo.

3.2 Fechas: contrato, no tipo nativo

Como JSON no tiene tipo fecha, debemos elegir representación. En el curso usaremos:

yyyy-MM-dd

Por ejemplo:

```
"fechaSolicitud": "2025-01-15"
```

Spring Boot configura soporte Java Time y, con nuestra baseline, `LocalDate` ya se representa normalmente como fecha ISO. Aun así, `@JsonFormat(pattern = "yyyy-MM-dd")` puede hacer explícita una decisión de contrato en el DTO.

Esto corrige una simplificación frecuente de material antiguo: no debemos enseñar que nuestro `LocalDate` necesariamente aparecerá como `[2025, 1, 15]`. La salida real depende de la configuración del `ObjectMapper` y de los módulos instalados.

3.3 null, ausencia y listas vacías no significan lo mismo

Compara:

```
{"observaciones": null}
```

con:

```
{}
```

y con:

```
{"documentos": []}
```

Son tres mensajes distintos:

- propiedad presente sin valor;
- propiedad ausente;
- colección presente y vacía.

`@JsonInclude(NON_NULL)` nos permitirá decidir que las propiedades nulas no aparezcan al serializar.

3.4 DTOs anidados

Las estructuras de negocio no siempre son planas. Podemos representar:

```
{
  "id": "12345",
  "solicitante": {
    "nombre": "Ana",
    "apellidos": "García López",
    "dni": "12345678A"
  }
}
```

Esto mantiene agrupados datos que conceptualmente pertenecen a una subestructura. Jackson puede serializar el DTO anidado recorriendo sus propiedades igual que hace con el objeto exterior.

3.5 Compatibilidad y propiedades desconocidas

En nuestra configuración de Spring Boot, el ObjectMapper auto-configurado está preparado para tolerar propiedades de entrada desconocidas en escenarios normales. Así, un cliente puede enviar:

```
{
  "id": "99999",
  "titular": "María López",
  "campoInexistente": "valor"
}
```

sin que campoInexistente tenga que convertirse en un atributo del DTO.

Hay que formular esto con precisión: **no es una ley universal de todo ObjectMapper Jackson creado en cualquier contexto**. Es comportamiento de la configuración que estamos utilizando y puede cambiar si configuramos FAIL_ON_UNKNOWN_PROPERTIES de otra forma.

Pregunta

¿Por qué omitir una propiedad nula no es necesariamente equivalente a enviarla con valor null?

Respuesta razonada

Porque algunos clientes distinguen “la propiedad no forma parte de esta representación” de “la propiedad está presente y su valor actual es nulo”. El contrato de la API debe decidir qué semántica quiere transmitir, especialmente en actualizaciones parciales y compatibilidad entre versiones.

Bloque 4 - Jackson

4.1 Serialización y deserialización

Jackson es la biblioteca que utilizaremos para convertir entre objetos Java y representaciones JSON.

Serialización:

objeto Java -> JSON

Deserialización:

JSON -> objeto Java

En Spring Boot con spring-boot-starter-web, no necesitamos añadir manualmente la dependencia principal de Jackson para este uso básico: llega dentro del stack web y Spring Boot prepara un ObjectMapper y conversores apropiados.

4.2 Cuando un controlador devuelve un DTO

Si un método @RestController devuelve un ExpedienteDTO, el flujo simplificado es:

1. el método devuelve el objeto Java;
2. Spring MVC selecciona un HttpMessageConverter apropiado;
3. para JSON, el conversor utiliza Jackson;
4. Jackson inspecciona las propiedades según su configuración y anotaciones;
5. produce JSON;
6. Spring escribe el cuerpo y declara el tipo de contenido.

No debemos imaginar que el controlador llama manualmente a `ObjectMapper.writeValueAsString(...)` en cada endpoint. La integración web automatiza ese paso.

4.3 Cuando usamos @RequestBody

Con:

```
@PostMapping("/eco")
public ExpedienteDTO eco(@RequestBody ExpedienteDTO dto) {
    return dto;
}
```

el recorrido inverso ocurre antes de entrar al método:

1. Spring observa Content-Type;
2. lee el cuerpo;
3. selecciona el conversor;
4. Jackson construye y rellena ExpedienteDTO;
5. el método recibe un objeto Java ya deserializado.

Si el JSON es inválido o un valor no puede convertirse al tipo esperado, la petición puede fallar antes de ejecutar el cuerpo del método.

4.4 Constructor y propiedades

Para un DTO mutable clásico es didácticamente útil proporcionar:

- constructor sin argumentos;
- getters;
- setters.

Jackson moderno admite estrategias adicionales —constructores anotados, records, visibilidad de campos, módulos—, pero no las necesitamos todavía. En este módulo emplearemos JavaBeans sencillos porque hacen visible el proceso de lectura y escritura.

4.5 Jackson no valida reglas de negocio

Que un JSON pueda convertirse a ExpedienteDTO no significa que los datos sean válidos para el negocio.

Por ejemplo:

```
{"importe": -999999}
```

puede ser sintácticamente JSON válido y convertible a Double, aunque una regla futura pudiera prohibir importes negativos.

Separaremos progresivamente:

- **parseo/conversión:** ¿puedo construir el objeto?;
- **validación de formato:** ¿cumple restricciones declarativas?;
- **regla de negocio:** ¿la operación tiene sentido en el dominio?

La validación formal se tratará más adelante; no debemos atribuir a Jackson responsabilidades que no le corresponden.

Pregunta

Si Jackson puede convertir un JSON a Java, ¿significa que los datos ya son correctos para guardarlos?

Respuesta razonada

No. Sólo demuestra que la representación puede convertirse a los tipos Java esperados. Una cadena puede ser demasiado larga, un DNI puede tener formato incorrecto o un identificador puede estar duplicado. Conversión y validación son problemas diferentes.

Bloque 5 - Anotaciones de Jackson

5.1 @JsonProperty

Permite definir el nombre JSON de una propiedad sin renombrar necesariamente el atributo Java:

```
@JsonProperty("id")
private String identificador;
```

Internamente seguimos utilizando `identificador`; externamente el contrato expone `id`.

Esto es útil para desacoplar parcialmente nombres internos y externos, aunque no debe convertirse en una excusa para usar nombres arbitrariamente contradictorios.

5.2 @JsonFormat

Podemos hacer explícito el formato de una fecha:

```
@JsonFormat(pattern = "yyyy-MM-dd")
private LocalDate fechaSolicitud;
```

En nuestra baseline la salida por defecto de `LocalDate` ya es normalmente ISO gracias a la configuración de Java Time. La anotación documenta y fija localmente la decisión.

5.3 @JsonInclude

A nivel de clase:

```
@JsonInclude(JsonInclude.Include.NON_NULL)
public class ExpedienteDTO {
    // ...
}
```

hace que propiedades nulas no se incluyan al serializar ese DTO.

No confundir con `@JsonIgnore`: `NON_NULL` depende del valor; una propiedad no nula sí aparecerá.

5.4 @JsonIgnore

Para una propiedad que no debe formar parte del contrato JSON:

```
@JsonIgnore
private String numeroSeguridadSocial;
```

Jackson la excluye de la representación según esa configuración. Esto puede ayudar a evitar exposición accidental, pero la seguridad de una API no debería depender únicamente de recordar una anotación: diseñar DTOs específicos de salida suele ser más seguro en sistemas reales. Esa evolución llegará en módulos posteriores.

5.5 Las anotaciones son parte del contrato

Cambiar:

```
@JsonProperty("id")
```

por:

```
@JsonProperty("identificador")
```

puede no romper la compilación Java y, sin embargo, romper clientes. Las anotaciones de serialización influyen en la interfaz externa y deben tratarse con la misma atención que una URL o un código HTTP.

Pregunta

¿Por qué @JsonIgnore no debe considerarse una solución completa para todos los problemas de exposición de datos?

Respuesta razonada

Porque sigue existiendo una única clase con responsabilidades internas y externas mezcladas. La anotación protege esa propiedad concreta en ese contrato Jackson, pero a medida que crece un sistema es más robusto diseñar DTOs de entrada y salida que contengan explícitamente sólo los datos permitidos. En este punto usamos @JsonIgnore para comprender Jackson; más adelante refinaremos el diseño.

Resumen del Punto 1.3

- JSON es un formato textual estructurado e independiente del lenguaje.
- Objetos, arrays, cadenas, números, booleanos y null forman su vocabulario básico.
- La sintaxis es estricta: comillas dobles, separadores y delimitadores importan.
- JSON no tiene un tipo fecha nativo; el formato de fecha es parte del contrato.
- Convenciones coherentes de nombres y estructuras reducen fricción para los clientes.
- Jackson serializa Java a JSON y deserializa JSON a Java.
- Spring MVC integra Jackson mediante conversores; los controladores no tienen que invocar ObjectMapper manualmente para casos normales.
- @RequestBody puede fallar antes de entrar al método si el cuerpo no puede convertirse.
- En nuestra configuración de Spring Boot, las propiedades desconocidas se toleran por defecto; no generalizamos esa afirmación a cualquier ObjectMapper.
- @JsonProperty, @JsonFormat, @JsonInclude y @JsonIgnore modifican el contrato de representación.
- Un JSON convertible no es necesariamente un dato válido para el negocio.

En el punto 1.4 utilizaremos estos DTOs para diseñar una API REST coherente: recursos, URLs, métodos, estados, versionado, filtrado y evolución del contrato.

Punto 1.4 - Diseño de APIs REST

Objetivos de aprendizaje

Al finalizar este punto serás capaz de:

1. Explicar REST como estilo arquitectónico y no como protocolo, librería o framework.
2. Modelar un dominio HTTP en términos de recursos, colecciones y representaciones.
3. Diseñar URLs centradas en sustantivos, consistentes, versionadas y fáciles de evolucionar.
4. Elegir el método HTTP según la semántica de la operación y razonar sobre seguridad e idempotencia.
5. Seleccionar códigos de estado que describan el resultado real de una petición.
6. Diferenciar identificadores de recurso en path de filtros, ordenación y paginación en query parameters.
7. Entender por qué una colección vacía no es un error y cómo deben expresarse los filtros opcionales.
8. Comprender el papel de la documentación y del versionado en la evolución de una API.
9. Aplicar estos principios a un recurso real Alumno sin adelantar todavía el CRUD completo del punto 1.5.

Bloque 1 - Qué es REST

1.1 REST es un estilo arquitectónico

REST significa *Representational State Transfer*. Roy Fielding lo describió en su tesis doctoral de 2000 como parte de un trabajo más amplio sobre arquitecturas de red. Para este curso importa una distinción fundamental: REST **no es HTTP**, no es una especificación de código, no es una dependencia Maven y no es una anotación de Spring.

REST es un conjunto de restricciones y principios para diseñar sistemas distribuidos. HTTP es el protocolo sobre el que normalmente materializamos esos principios cuando construimos una API web.

En nuestro proyecto esto significa que Spring MVC puede recibir una petición aunque nuestra URL esté mal diseñada. Que el código funcione técnicamente no garantiza que la API tenga un diseño REST coherente.

1.2 Recursos y representaciones

La unidad conceptual no es la operación, sino el **recurso**. Un alumno, un expediente o una colección de alumnos son recursos con identidad. La URL identifica el recurso y el método HTTP expresa qué queremos hacer con él.

```
/api/v1/alumnos      -> colección de alumnos  
/api/v1/alumnos/1    -> alumno identificado por 1
```

El servidor no envía el objeto Java en sí. Envía una **representación** del estado del recurso. En este curso esa representación será normalmente JSON.

Esta separación permite que el modelo interno evolucione sin obligar a que el cliente conozca clases Java. El contrato público es HTTP + representación, no la estructura de memoria del servidor.

1.3 Restricciones que dan sentido a REST

La descripción clásica de REST incluye varias restricciones arquitectónicas. Para empezar debemos reconocer al menos estas ideas:

- **cliente-servidor:** responsabilidades separadas;
- **sin estado de sesión en la petición (*stateless*):** cada petición contiene la información necesaria para procesarse;
- **cacheable:** las respuestas pueden declarar cuándo son reutilizables;
- **interfaz uniforme:** recursos, métodos y representaciones siguen reglas consistentes;
- **sistema en capas:** el cliente no necesita conocer toda la topología interna;
- **código bajo demanda:** restricción opcional en la definición clásica.

No vamos a implementar todas estas capacidades ahora. Sí utilizaremos su consecuencia práctica: una API debe ser predecible. El cliente no debería tener que memorizar una regla distinta para cada recurso.

Pregunta

¿Por qué `/api/getAlumnos` es una URL menos REST que `/api/v1/alumnos` aunque ambas puedan devolver exactamente el mismo JSON?

Respuesta razonada

Porque la primera modela una acción dentro de la URL y duplica información que ya aporta el método HTTP. La segunda identifica el recurso `alumnos`; después `GET` expresa que queremos consultarlo. Separar identidad del recurso y semántica de la operación hace que el contrato sea más uniforme y extensible.

Bloque 2 - Diseño de URLs

2.1 La URL identifica; el método actúa

Una URL REST debe permitir reconocer qué recurso representa. Conviene utilizar sustantivos, normalmente en plural para colecciones:

```
GET /api/v1/alumnos
GET /api/v1/alumnos/1
POST /api/v1/alumnos
```

Evita convertir la ruta en una llamada a método remota:

```
/api/getAlumnos
/api/crearAlumno
/api/eliminarAlumno?id=1
```

Estas rutas pueden funcionar, pero trasladan al URI una semántica que HTTP ya proporciona mediante GET, POST o DELETE.

2.2 Colecciones, elementos y subrecursos

El patrón colección/elemento es una de las convenciones más útiles:

```
/api/v1/alumnos          -> colección
/api/v1/alumnos/{id}     -> elemento
/api/v1/alumnos/{id}/documentos -> subrecurso relacionado
```

El identificador va en el path cuando forma parte de la identidad del recurso. En cambio, una condición como `curso=5º Primaria` no identifica un alumno concreto: modifica la consulta sobre la colección y encaja mejor como query parameter.

2.3 Convenciones de nomenclatura

Un contrato consistente suele utilizar:

- minúsculas;
- sustantivos en plural para colecciones;
- guiones medios si un nombre necesita varias palabras;
- ausencia de extensiones como `.json` en la ruta;
- jerarquías sólo cuando expresan una relación real.

Por ejemplo:

```
/api/v1/tipos-alumno
```

es preferible a:

```
/api/v1/Tipos_Alumno.json
```

2.4 Versionado

Nuestro curso utiliza el prefijo `/api/v1/`. No es la única estrategia posible, pero hace visible qué contrato consume el cliente:

```
/api/v1/alumnos
```

El objetivo del versionado no es crear una versión por cada pequeño cambio. Sirve para gestionar cambios incompatibles de forma controlada. Añadir un campo opcional suele ser compatible; cambiar el significado de un campo existente o eliminarlo puede no serlo.

Pregunta

¿Por qué `GET /api/v1/alumnos?id=1` no expresa tan bien la identidad como `GET /api/v1/alumnos/1`?

Respuesta razonada

Porque el primer formato trata `id=1` como criterio de consulta sobre una colección. El segundo identifica directamente el recurso alumno 1. Los query parameters son adecuados para modificar una consulta —por ejemplo filtros—, mientras que el path representa la identidad o jerarquía del recurso.

Bloque 3 - Métodos HTTP y códigos de estado

3.1 La semántica de los métodos

HTTP ya define verbos con significado. Utilizarlos correctamente evita inventar contratos propios:

Método	Intención habitual	¿Seguro?	¿Idempotente?
GET	consultar	sí	sí
POST	crear/procesar	no	no, en general
PUT	reemplazar completamente	no	sí
PATCH	modificar parcialmente	no	no, en general
DELETE	eliminar	no	sí

Seguro significa que la intención de la operación no es modificar el estado del servidor. **Idempotente** significa que repetir la misma operación produce el mismo estado final que ejecutarla una vez.

La idempotencia importa especialmente ante reintentos. Repetir un GET después de un problema de red no debería crear nada. Repetir un POST puede producir un segundo recurso si la API no introduce mecanismos adicionales.

3.2 El código de estado forma parte del contrato

El cuerpo no es toda la respuesta. El código HTTP permite al cliente interpretar el resultado sin analizar primero un texto arbitrario.

En este punto utilizaremos principalmente:

- 200 OK: consulta correcta;
- 201 Created: creación correcta;
- 400 Bad Request: petición que no puede convertirse/procesarse por su formato;
- 404 Not Found: recurso individual inexistente;
- 405 Method Not Allowed: ruta existente pero método no admitido;
- 415 Unsupported Media Type: representación enviada con un tipo no soportado.

Más adelante aparecerán otros códigos según nuevas necesidades.

3.3 ResponseEntity cuando necesitamos controlar HTTP

Devolver directamente un DTO es cómodo. Spring serializa el objeto y, si todo va bien, responde con 200. Esa convención es apropiada para muchos GET, pero no expresa una creación correctamente: un POST que crea un recurso debe comunicar 201 Created.

`ResponseEntity<T>` permite controlar código, cabeceras y cuerpo:

```
return ResponseEntity.status(HttpStatus.CREATED).body(dto);
```

También permite representar ausencia:

```
return ResponseEntity.notFound().build();
```

No usaremos `ResponseEntity` por costumbre en todos los métodos; lo utilizaremos cuando el contrato HTTP necesite información que un cuerpo por sí solo no expresa.

Pregunta

¿Por qué no basta con devolver null y dejar que el cliente deduzca que un alumno no existe?

Respuesta razonada

Porque null no expresa por sí mismo el significado HTTP. Un cliente necesita distinguir un recurso inexistente de una respuesta correcta cuyo contenido admita ausencia. 404 Not Found hace explícito el resultado en la capa correcta del protocolo.

Bloque 4 - Paginación, filtrado y ordenación

4.1 Los query parameters modifican una consulta

Una colección puede crecer mucho. El cliente suele necesitar seleccionar una vista concreta sin crear una ruta nueva para cada combinación.

Ejemplos:

```
GET /api/v1/alumnos?curso=5%C2%BA%20Primaria
```

```
GET /api/v1/alumnos?page=0&size=20
```

```
GET /api/v1/alumnos?sort=apellidos,asc
```

El recurso sigue siendo la colección alumnos; los parámetros modifican cómo se consulta o representa esa colección.

4.2 Filtrado

Un filtro expresa una condición, no una identidad:

```
?curso=5º Primaria
```

Si el filtro es opcional, ausencia de parámetro significa “no filtrar por ese criterio”. Una colección que no contiene coincidencias debe poder representarse naturalmente como [] con una petición válida; no es equivalente a que la ruta no exista.

4.3 Paginación

Una convención habitual utiliza page y size:

```
?page=0&size=20
```

El cliente indica qué ventana quiere consultar. Aunque en 1.4 sólo implementaremos el filtro curso, necesitamos comprender la convención porque el diseño REST debe anticipar colecciones que crecerán.

4.4 Ordenación

La ordenación suele expresarse también con query parameters:

```
?sort=fecha,desc
```

Puede combinarse con filtrado y paginación sin multiplicar endpoints:

```
/api/v1/becas?estado=EN_TRAMITE&sort=fecha,desc&page=0&size=20
```

La clave es la consistencia: si curso filtra alumnos mediante query parameter, otros recursos deberían seguir una regla equivalente para filtros comparables.

Pregunta

¿Por qué un filtro que no encuentra resultados debería devolver [] con 200 en lugar de 404?

Respuesta razonada

Porque la colección y la ruta sí existen, y la consulta es válida. El resultado de aplicar el criterio es una colección con cero elementos. 404 expresa que no se encontró el recurso identificado por la URL, no que una consulta válida sobre una colección produzca cero coincidencias.

Bloque 5 - Documentación y evolución

5.1 El contrato debe poder descubrirse

Una API necesita explicar, como mínimo:

- qué recursos y endpoints existen;
- métodos HTTP admitidos;
- parámetros de path y query;
- estructura de los cuerpos JSON;
- códigos de éxito y error;
- ejemplos relevantes.

Puede hacerse con documentación manual o con especificaciones que permitan generar documentación, como OpenAPI. En este punto no añadiremos todavía una dependencia documental: primero necesitamos aprender a diseñar correctamente el contrato que después documentaremos.

5.2 Documentar comportamiento, no sólo rutas

Una lista de URLs es insuficiente. Por ejemplo, para `POST /api/v1/alumnos` necesitamos saber que la entrada es JSON y que la creación correcta responde 201. Para `GET /api/v1/alumnos/{id}` debemos documentar tanto 200 como 404.

El código puede compilar aunque la documentación esté equivocada. Por eso los ejemplos ejecutables y los tests de contrato son importantes: reducen la distancia entre lo documentado y lo que el servidor hace realmente.

5.3 Evolución compatible

Una API publicada tiene consumidores. Cambiarla obliga a pensar en compatibilidad:

- añadir capacidades opcionales suele ser menos disruptivo;
- renombrar o eliminar campos puede romper clientes;
- cambiar códigos de estado altera lógica del consumidor;
- cambiar una URL obliga a actualizar integraciones.

El versionado es una herramienta para gestionar cambios incompatibles, no una excusa para no diseñar con cuidado.

5.4 Diseño antes de código

La práctica de 1.4 empieza deliberadamente sin programar. Primero enumeraremos recursos, rutas, métodos y resultados. Sólo después crearemos `AlumnoDTO` y `AlumnoController`.

Ese orden importa: si dejamos que el primer método Java que se nos ocurra determine la API, convertimos decisiones internas en contrato público por accidente.

Pregunta

¿Qué riesgo existe si publicamos una API y luego cambiamos libremente nombres de campos, rutas y códigos de estado?

Respuesta razonada

Que los clientes ya integrados dejan de poder interpretar el contrato. Una API es una frontera entre sistemas; evolucionarla exige tratar compatibilidad, documentación y versionado como parte del diseño, no como tareas posteriores al código.

Resumen del punto 1.4

- REST es un estilo arquitectónico centrado en recursos y representaciones.
- Las URLs identifican recursos; los métodos HTTP expresan operaciones.
- Colecciones en plural, minúsculas y versionado coherente facilitan un contrato predecible.
- GET, POST, PUT, PATCH y DELETE tienen semántica distinta, incluida seguridad e idempotencia.
- Los códigos HTTP describen el resultado real; 200, 201 y 404 no son intercambiables.
- Los identificadores individuales encajan en el path; filtros, paginación y ordenación encajan en query parameters.
- Una consulta válida sin coincidencias devuelve una colección vacía, no un 404.
- Documentación y evolución forman parte del contrato de una API.
- En la práctica construiremos AlumnoDTO y una primera API de alumnos sin adelantar el CRUD completo de 1.5.

Punto 1.5 - Primer CRUD con DTOs

Objetivos de aprendizaje

Al finalizar este punto serás capaz de:

1. Explicar el ciclo CRUD y relacionar Create/Read/Update/Delete con POST/GET/PUT-PATCH/DELETE.
2. Diferenciar actualización completa con PUT de actualización parcial con PATCH.
3. Explicar idempotencia y seguridad en el contexto de operaciones CRUD.
4. Elegir códigos HTTP coherentes para creación, consulta, actualización y borrado.
5. Entender qué significa persistencia en memoria y qué limitaciones tiene frente a una capa de datos real.
6. Implementar un CRUD completo sobre AlumnoDTO manteniendo el identificador de la URL como identidad del recurso.
7. Explicar por qué un DELETE puede devolver 204 la primera vez y 404 la segunda sin dejar de ser idempotente en estado final.
8. Reconocer la fragilidad pedagógica de un PATCH basado en Map<String, Object>.
9. Mantener filtrado y ordenación como parámetros opcionales de la colección.
10. Identificar qué responsabilidades del controlador deberán extraerse a servicio y repositorio en los módulos siguientes.

Bloque 1 - El ciclo CRUD

1.1 Qué significa CRUD

CRUD resume cuatro operaciones básicas sobre un recurso:

Create -> crear
Read -> consultar
Update -> actualizar
Delete -> eliminar

En una API REST solemos proyectarlas sobre HTTP así:

Operación	Método HTTP habitual	Recurso
Create	POST	colección

Operación	Método HTTP habitual	Recurso
Read colección	GET	colección
Read individual	GET	elemento
Update completo	PUT	elemento
Update parcial	PATCH	elemento
Delete	DELETE	elemento

La fuente insiste en una idea importante: el CRUD es el esqueleto de muchos sistemas de gestión. Filtros, seguridad, persistencia real, validación y reglas de negocio se apoyan después sobre estas operaciones básicas.

En nuestro recurso Alumno:

```
GET    /api/v1/alumnos
GET    /api/v1/alumnos/{id}
POST   /api/v1/alumnos
PUT    /api/v1/alumnos/{id}
PATCH /api/v1/alumnos/{id}
DELETE /api/v1/alumnos/{id}
```

No son seis recursos distintos. Son dos identidades principales —colección y elemento— sobre las que aplicamos diferentes métodos.

1.2 Crear no es sólo devolver el cuerpo

En 1.4 nuestro POST devolvía 201, pero no modificaba la colección. Eso servía para separar dos conceptos:

- **contrato HTTP:** status y representación devuelta;
- **efecto sobre el estado del servidor:** persistencia.

En 1.5 conectamos ambas cosas. Cuando el POST sea correcto deberá:

1. recibir un AlumnoDTO;
2. asignarle un identificador si no lo tiene;
3. añadirlo a la colección mutable;
4. responder 201 con el recurso creado.

La lista en memoria sustituye temporalmente a una base de datos. Es útil para aprender el ciclo CRUD, pero desaparece al reiniciar la aplicación y no resuelve concurrencia, transacciones ni persistencia durable.

1.3 Read: colección e individuo no significan lo mismo

El GET de colección devuelve 200 aunque el resultado sea []. El recurso colección existe; simplemente no hay elementos que satisfagan la consulta.

El GET individual puede devolver 404 porque la URL identifica un recurso concreto que quizá no exista.

La diferencia no es estética. Permite al cliente distinguir:

```
colección válida sin resultados -> 200 + []
recurso individual inexistente -> 404
```

Pregunta

¿Un POST que responde 201 pero no añade el alumno a ningún almacenamiento completa realmente la operación Create?

Respuesta razonada

No en el sentido funcional del CRUD. El status puede afirmar creación, pero si una consulta posterior no puede recuperar el recurso, falta el efecto de persistencia. En 1.4 aislamos deliberadamente el contrato HTTP; en 1.5 conectamos contrato y cambio de estado.

Bloque 2 - PUT en detalle

2.1 PUT reemplaza una representación completa

La fuente define PUT como “reemplaza el recurso existente con esta representación”, no como “actualiza sólo los campos enviados”.

Si el recurso tiene estos campos:

id, nombre, apellidos, dni, fechaNacimiento, curso, documentos

un PUT conceptual debe representar el nuevo estado completo. Si queremos modificar sólo curso, PATCH expresa mejor la intención.

En nuestro ejercicio el identificador de la URL será la autoridad:

PUT /api/v1/alumnos/3

Aunque el cuerpo trajera otro id, el recurso que se está reemplazando es el identificado por /3. Por eso el controlador fijará explícitamente:

```
dto.setIdentificador(id);
```

antes de sustituir el elemento de la colección.

2.2 Idempotencia de PUT

PUT es idempotente: repetir la misma petición deja el recurso en el mismo estado final que ejecutarla una sola vez.

Si enviamos cinco veces:

```
PUT /api/v1/alumnos/3
{ ... representación completa ... }
```

no debemos crear cinco alumnos ni acumular cambios. Cada ejecución vuelve a establecer el mismo estado del recurso 3.

Esto es especialmente útil ante reintentos de red: si el cliente no sabe si recibió la respuesta, puede repetir la operación sin duplicar el recurso.

2.3 PUT existente e inexistente

Para nuestro CRUD introductorio utilizaremos:

```
recurso existente -> 200 OK + recurso actualizado
recurso inexistente -> 404 Not Found
```

La fuente admite también 204 para una actualización correcta sin cuerpo, pero en este ejercicio devolveremos 200 para poder observar el recurso resultante.

Pregunta

¿Por qué no usamos `alumnos.add(dto)` para implementar PUT?

Respuesta razonada

Porque PUT opera sobre una identidad existente y debe reemplazar ese recurso, no crear otro elemento al final de la colección. Añadir siempre produciría duplicados y rompería la idempotencia esperada.

Bloque 3 - PATCH y DELETE en detalle

3.1 PATCH modifica parcialmente

PATCH expresa una modificación parcial: los campos no enviados conservan su valor actual. La fuente lo contrasta explícitamente con PUT.

En este primer ejercicio recibiremos:

```
Map<String, Object> cambios
```

Y sólo admitiremos campos concretos que sepamos convertir con seguridad, por ejemplo:

```
nombre  
apellidos  
dni  
curso
```

Este Map es una simplificación pedagógica, no un diseño final recomendado. Tiene varias limitaciones:

- pierde tipado estático;
- obliga a comprobar nombres manualmente;
- puede requerir conversiones explícitas;
- dificulta validación y documentación.

Más adelante podremos usar DTOs específicos de entrada y validación formal.

3.2 Idempotencia de PATCH: cuidado con las generalizaciones

La fuente indica correctamente que PATCH **no es idempotente en general**.

Sin embargo, una operación concreta como:

```
{"curso": "6º Primaria"}
```

implementada como simple asignación sí puede dejar el mismo estado final al repetirse. La regla importante es no concluir que **todo PATCH** es idempotente. Un PATCH como “incrementa el contador en 1” no lo sería.

3.3 DELETE elimina el recurso

DELETE trabaja sobre la URL individual:

```
DELETE /api/v1/alumnos/3
```

En nuestro almacenamiento en memoria utilizaremos `removeIf`:

```
boolean eliminado = alumnos.removeIf(  
    a -> a.getIdentificador().equals(id));
```

Resultado:

```
se eliminó algo -> 204 No Content  
no existía      -> 404 Not Found
```

La fuente propone precisamente ese comportamiento y pregunta si sigue siendo idempotente cuando la segunda llamada devuelve 404.

Sí: la idempotencia describe el **estado final**. Después de la primera y de la segunda llamada el recurso está ausente. El status puede ser diferente porque la segunda petición observa un estado previo diferente.

Pregunta

Si DELETE devuelve 204 la primera vez y 404 la segunda, ¿por qué seguimos considerándolo idempotente?

Respuesta razonada

Porque ambas secuencias terminan con el mismo estado del servidor: el recurso no existe. Idempotencia no significa “misma respuesta HTTP en cada repetición”, sino “mismo efecto final sobre el estado”.

Bloque 4 - Códigos de estado en el CRUD

4.1 Tabla de contrato

La fuente resume los códigos habituales del CRUD así: GET colección 200; GET individual 200/404; POST 201; PUT y PATCH 200 o 204/404; DELETE 204/404.

En nuestro ejercicio fijamos:

Operación	Éxito	Recurso inexistente	Formato/tipo inválido
GET colección	200	—	—
GET individual	200	404	—
POST	201	—	400 / 415
PUT	200	404	400 / 415
PATCH	200	404	400 / 415
DELETE	204	404	—

También veremos 405 Method Not Allowed cuando la ruta existe pero el método solicitado no está mapeado.

4.2 400 no es lo mismo que 415

Estos dos errores aparecen antes de que nuestro método pueda trabajar correctamente con el objeto:

- 400 Bad Request: JSON mal formado o no convertible al tipo esperado;
- 415 Unsupported Media Type: el cliente envía un cuerpo con un tipo que el endpoint no acepta, por ejemplo text/plain cuando esperamos JSON.

No debemos escribir lógica manual para producirlos si Spring MVC/Jackson ya pueden detectarlos en la capa correcta.

4.3 404 y 405 tampoco son equivalentes

GET /api/v1/alumnos/999 -> 404
POST /api/v1/alumnos/1 -> 405

En el primer caso existe el patrón de endpoint, pero el recurso 999 no. En el segundo la URL individual está reconocida para otros métodos, pero POST no está permitido allí.

4.4 Consistencia

La fuente subraya que una API predecible debe utilizar los mismos códigos para resultados equivalentes.

No tendría sentido que un GET individual inexistente devolviera 404 en un recurso y 200 con null en otro sin una razón contractual explícita. La consistencia permite al cliente construir manejo genérico de errores.

Pregunta

¿Por qué un cliente se beneficia de que todos los recursos usen 404 de forma consistente cuando no existe un elemento individual?

Respuesta razonada

Porque puede implementar una única política para “recurso no encontrado”. Si cada endpoint inventa un cuerpo o status distinto, el cliente necesita excepciones específicas y el contrato se vuelve más difícil de integrar.

Bloque 5 - Consolidación del Módulo 1

5.1 Lo que ya sabemos construir

Al cerrar M1 habremos recorrido una cadena completa:

Spring Boot y auto-configuración

↓

cliente-servidor + HTTP

↓

JSON + Jackson

↓

diseño REST

↓

CRUD con DTOs

Eso significa que ya podemos explicar no sólo que un endpoint “funciona”, sino por qué existe, qué contrato HTTP tiene, cómo se representa en JSON y cómo cambia el estado en memoria.

5.2 Lo que todavía no hemos resuelto

Nuestro AlumnoController final de M1 será deliberadamente monolítico. Tendrá:

- almacenamiento en memoria;
- generación pedagógica de IDs;
- búsqueda;
- modificación;
- borrado;
- manejo HTTP;
- filtrado y ordenación.

Funciona como ejercicio, pero reúne demasiadas responsabilidades. Esa limitación prepara el siguiente módulo: separar controlador, servicio y repositorio.

Tampoco tenemos todavía:

- base de datos;
- transacciones;
- validación declarativa de entrada;
- reglas de negocio robustas;
- tratamiento global de excepciones;
- seguridad;
- concurrencia controlada;
- generación de IDs de producción.

5.3 La generación de ID es intencionadamente simple

La fuente pedagógica utiliza una idea equivalente a `size()+1` para que el nuevo alumno obtenga el ID 3. La conservaremos como simplificación visible, pero con una advertencia: después de borrar elementos puede producir colisiones.

Una versión algo más robusta dentro del mismo ejercicio puede calcular el máximo ID numérico actual y sumar uno. Aun así sigue siendo almacenamiento local, no una estrategia válida para concurrencia o producción.

5.4 Filtro y ordenación siguen siendo consulta

El último reto combinará:

```
?curso=...  
?sort=nombre  
?sort=apellidos
```

Ambos siguen modificando la vista de la colección, por lo que pertenecen a query parameters.

La edición corrige una contradicción del ejemplo histórico: si sort es opcional, no usaremos default-Value="nombre", porque eso ordenaría siempre aunque el cliente no hubiera solicitado ordenación. Sin sort, conservaremos el orden actual de la colección.

5.5 El final de M1 es el punto de partida de M2

El objetivo no es terminar con un controlador “perfecto”. El objetivo es llegar a un estado funcional cuya siguiente debilidad resulte evidente: la lógica del CRUD no debería vivir toda dentro del controlador.

Eso permitirá que M2 introduzca capas de aplicación sobre algo que ya comprendemos funcionalmente.

Pregunta

¿Por qué tiene sentido construir primero un CRUD sencillo dentro del controlador si luego vamos a refactorizarlo?

Respuesta razonada

Porque primero aislamos el comportamiento observable y el contrato HTTP. Cuando después movamos lógica a servicio y repositorio podremos comprobar que el comportamiento no cambia. Refactorizar resulta mucho más comprensible cuando sabemos exactamente qué estamos preservando.

Resumen del punto 1.5

- CRUD significa Create, Read, Update y Delete.
- POST crea; GET consulta; PUT reemplaza; PATCH modifica parcialmente; DELETE elimina.
- PUT es idempotente y debe representar una actualización completa.
- PATCH es parcial y no debe asumirse idempotente de forma universal.
- DELETE puede responder 204 y después 404 sin perder idempotencia de estado final.
- Los códigos HTTP forman parte del contrato, no son decoración.
- El almacenamiento en una ArrayList sirve para aprender, pero no es persistencia de producción.
- El POST final debe guardar el alumno y devolver 201.
- PUT/PATCH/DELETE deben devolver 404 si el ID no existe.
- El listado final conserva filtrado y añade ordenación opcional.
- El controlador resultante será funcional, pero deliberadamente listo para ser refactorizado en M2.